Enerza — Explicação do projeto e dos diagramas

EXPO2026 · Projeto do 4º semestre · material de estudo para a apresentação

1. O que é o projeto

O **Enerza** é um painel de casa inteligente. Um programa em **Java** funciona como servidor web: entrega as páginas HTML e atende as requisições do navegador. Ele usa três serviços:

- **MySQL**: guarda usuários, sessões, famílias e aparelhos.
- **Tuya (Smart Life)**: nuvem que liga e desliga tomadas e lâmpadas.
- **Spotify**: controla a música.

O padrão é sempre o mesmo: o **navegador pede**, o **servidor decide** e responde.

2. Como ler cada diagrama

Casos de uso (página 1 do PDF)

Mostra **quem faz o quê**. Os bonecos são os atores; as elipses são as funções do sistema.

Ator	O que faz
Visitante	Ainda sem login. Só se cadastra e faz login.
Membro	Usa o painel: Spotify, aparelhos, timers, gastos de energia, temperatura, troca de senha.
Administrador	Faz tudo o que o Membro faz e também adiciona/remove aparelhos, troca o cômodo e gere a família.

- A seta com **triângulo vazio** (generalização) significa “o Administrador é um tipo de Membro”.
- **«extend»**: “Ligar tudo” e “Ajustar lâmpada” são extras opcionais de “Ligar/desligar aparelho”.

Diagrama de classes (página 2)

Mostra **a estrutura do código Java**. Cada caixa é uma classe: atributos em cima, métodos embaixo.

Classe	Papel
`Servidor`	Ponto de entrada (`main`). Cria o servidor web e distribui cada requisição para a classe certa.
`AuthService`	Cadastro, login, logout, troca de senha e família.
`DispositivoService`	Lista, adiciona, remove, liga e desliga aparelhos.
`TuyaClient`	Conversa com a API da nuvem Tuya.
`LuzTuya`	Cor, brilho e temperatura do branco da lâmpada.
`SpotifyService`	Login no Spotify e controles de música.
`AuthService.Usuario`	Classe interna: id, nome, e-mail, família e cargo.
`DispositivoService.Linha`	Classe interna: uma linha da tabela de aparelhos.

- **Losango preenchido**: composição (a classe interna pertence à externa).
- **Seta tracejada**: "usa".
- Os bonecos Spotify e Nuvem Tuya são os serviços externos.

Diagramas de sequência (páginas 3 e 4)

Mostram **a ordem no tempo** de uma ação. Cada coluna é um participante e o tempo corre de cima para baixo. **Seta cheia** = pedido; **seta tracejada** = resposta.

Login:

1. O usuário digita e-mail e senha.
2. O navegador envia `POST /api/auth/login`.
3. O servidor busca o usuário no MySQL.
4. O servidor confere a senha e cria a sessão.
5. O servidor devolve o cookie e o navegador abre o painel.

Ligar aparelho:

1. O membro clica no botão.
2. O navegador envia o pedido.
3. O servidor confere o login.
4. O servidor manda o comando à nuvem Tuya.
5. A Tuya confirma e a tela atualiza.

3. O código, parte por parte

Estrutura de pastas

```
Enerza/
├── src/               código Java (o servidor)
├── html/              páginas HTML, css/ e script/
├── config.properties  porta, MySQL, Tuya, Spotify
└── executar.bat       compila e liga o servidor
```

Servidor.java — a “recepção”

Cria um servidor HTTP na porta 8080 e define as rotas:

```
server.createContext("/api/auth/", auth::tratar);              // pública (login
precisa ser acessível)
server.createContext("/api/spotify/", protegido(spotify::tratar));
server.createContext("/api/dispositivos/", protegido(dispositivos::tratar));
server.createContext("/", Servidor::arquivoEstatico);          // entrega HTML/CSS/JS
```

O método `protegido(...)` é um “segurança”: pergunta `auth.emailLogado(ex)`. Se ninguém estiver logado, responde **401** (APIs) ou redireciona para o login.

AuthService.java — contas e sessões

- **Cadastro:** valida nome, e-mail e senha (mínimo de 6 caracteres) e grava no banco.
- **Senha:** nunca em texto. Usa **PBKDF2 com sal** (o sal é aleatório por usuário, então senhas iguais geram hashes diferentes). No banco ficam `senha_sal` e `senha_hash`.
- **Login:** busca o usuário pelo e-mail, calcula o hash da senha digitada com o sal guardado e compara com o do banco. Se for igual, inicia a sessão. O hash é calculado mesmo se o e-mail não existir, para o tempo de resposta não revelar quais e-mails estão cadastrados.
- **Sessão:** token aleatório de 32 bytes enviado como **cookie** `HttpOnly` (o JavaScript do navegador não o lê). No banco fica só o hash do token, na tabela `sessoes`, com data de expiração (30 dias em “manter conectado”). Por isso reiniciar o servidor não derruba o login.
- **Família e cargo:** cada usuário pertence a uma família com cargo `admin` ou `membro`.

DispositivoService.java — aparelhos

Cada aparelho fica na tabela `dispositivos` (nome, ícone, cômodo, `tuya_id`, potência em watts) e pertence à **família**, então todos os membros veem os mesmos.

O método `ligar()`:

1. Busca o aparelho **somente da família do usuário logado** (impede mexer no aparelho de outra casa).
2. Chama `tuya.comando(...)`.
3. Responde `{"ok":true,"ligado":...}`.
4. Se a Tuya rejeitar o código do interruptor (erro 2008), descobre o código certo, tenta de novo e salva no banco.

TuyaClient.java — nuvem Tuya

Pede primeiro um token de acesso. Cada chamada leva uma **assinatura HMAC-SHA256** (client id, token, horário e conteúdo, usando o segredo). A Tuya confere essa assinatura para saber que o pedido veio do seu projeto.

LuzTuya.java e SpotifyService.java

- **LuzTuya:** lê o que a lâmpada suporta (cor RGB, brilho, temperatura do branco) e só mostra no painel os controles que ela tem.

- **SpotifyService:** faz o login OAuth e guarda o token **só no servidor**. O navegador pede, por exemplo, `/api/spotify/play` e o servidor repassa ao Spotify.

Front-end (html/script/)

Arquivo	Função
`login.js`	Envia e-mail e senha com `fetch("/api/auth/login")`. Se der certo, vai para `inicio.html`; se não, mostra o erro.
`controle.js`	Painel de aparelhos (liga/desliga, roda de cores, brilho). Se a resposta for 401, manda para `login2.html`. Atualiza a cada ~10 s.
`gastos.js`	Soma o consumo por cômodo (aparelho desligado conta 0 W).
`timer.js`	Timers da cozinha e do banheiro (rodam só no navegador).
`clima.js`	Temperatura pela Open-Meteo, direto do navegador.
`script.js`	Tela inicial e player do Spotify.

4. O caminho completo de um clique

Ao apertar “ligar” na lâmpada:

1. `controle.js` envia `POST /api/dispositivos/ligar`; o cookie vai junto automaticamente.
2. `Servidor` vê que a rota é protegida e chama `auth.emailLogado()`, que procura o hash do cookie na tabela `sessoes`.
3. Logado: `DispositivoService.ligar()` busca o aparelho da sua família no MySQL.
4. `TuyaClient` assina e envia o comando à nuvem Tuya.
5. A Tuya liga a lâmpada e responde; o servidor devolve o JSON.
6. `controle.js` atualiza o botão na tela.

5. Pontos para falar na apresentação

- **Segurança:** senha com hash e sal, cookie HttpOnly, token do Spotify só no servidor, rotas protegidas por sessão.
- **Permissões:** o cargo `admin` ou `membro` define quem adiciona e remove aparelhos.
- **Integração real:** o servidor conversa com três serviços externos (MySQL, Tuya e Spotify).
- **Arquitetura:** navegador, servidor Java e serviços externos, cada um com um papel claro.

Ressalva: esta explicação foi feita lendo os trechos principais do código (login, `ligar()`, rotas protegidas, assinatura da Tuya), não todas as linhas. Alguns passos dos diagramas são resumos (por exemplo, “confere a senha e cria a sessão” em um único passo). Antes de apresentar, vale conferir qualquer detalhe específico no código.